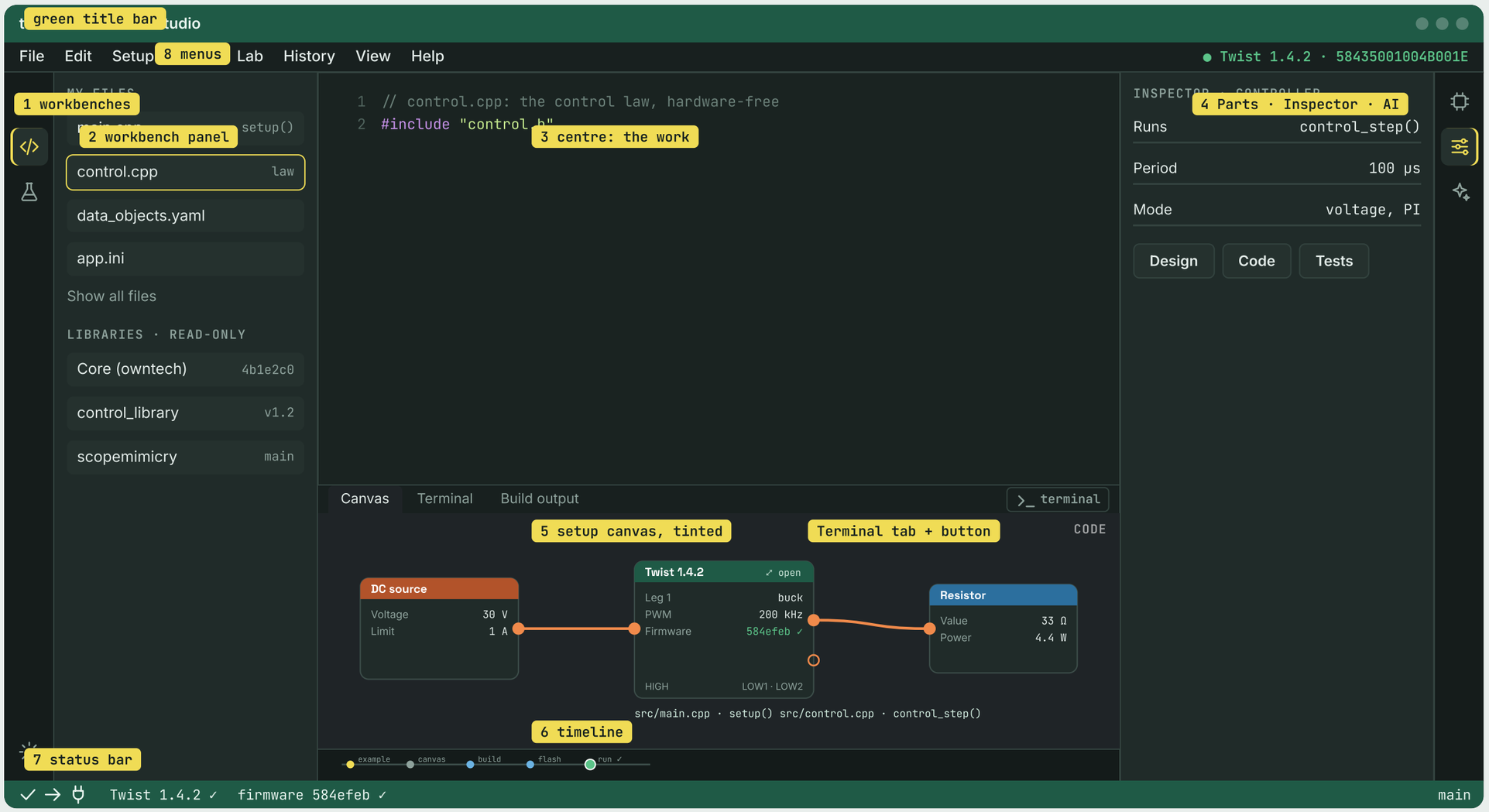
OWNSTUDIO · SLICE 0 · STORYBOARDS · FOR LUIZ'S REVIEW

OwnStudio 2.0, screen by screen

Luiz's nine-step objective (SPEC §2), drawn as the windows a colleague would see, from the Start page to a second run. Each frame says which slice builds it. Grey-green is the window; yellow marks what the frame is about. Values are the buck example's: 30 V supply, Twist 1.4.2, 33 Ω on LOW1.

0 The window, part by part · 1 Start: choose a board · 2 Drag the source, the Twist and the load · 3 Wire them; a wrong wire does not snap · 4 Open the Twist and look at its parts · 5 Load an example: it lands on its own branch · 6 Code: the student's files, with real language support · 7 Build and flash, with real progress · 8 The Lab checks before Run turns green · 9 A simple procedure: switch on and capture · 10 The result: interactive, zoomed on the step · 11 Open in PulsimGui: the simulator's circuit · 12 Simulate: the simulation over the measurement · 13 Design: change a parameter on a live sheet · 14 Run again; go back in time; branch · 15 The eight menus · 16 The AI: your own agent, and the takeover button



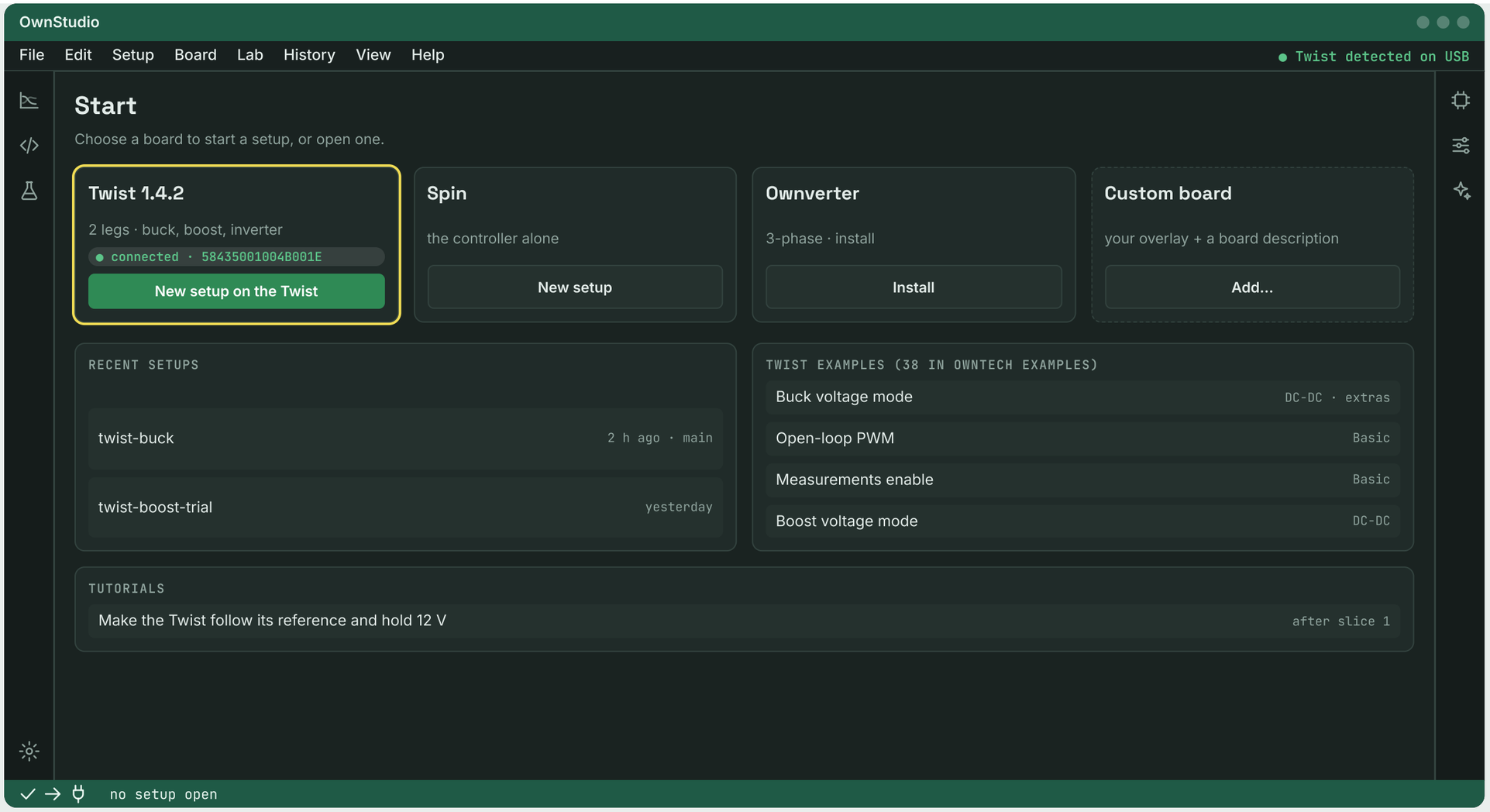
0

The window, part by part

OBJECTIVE: ALL STEPS

slice 1

- Workbenches on the left icon bar; their panel beside it.
- Centre: the work itself.
- Right icon bar: Parts, Inspector, AI.
- Bottom: the setup canvas (tinted per workbench) with a Terminal tab and button, and the timeline under it.
- Status bar: board, firmware, checklist, progress. Green title bar.



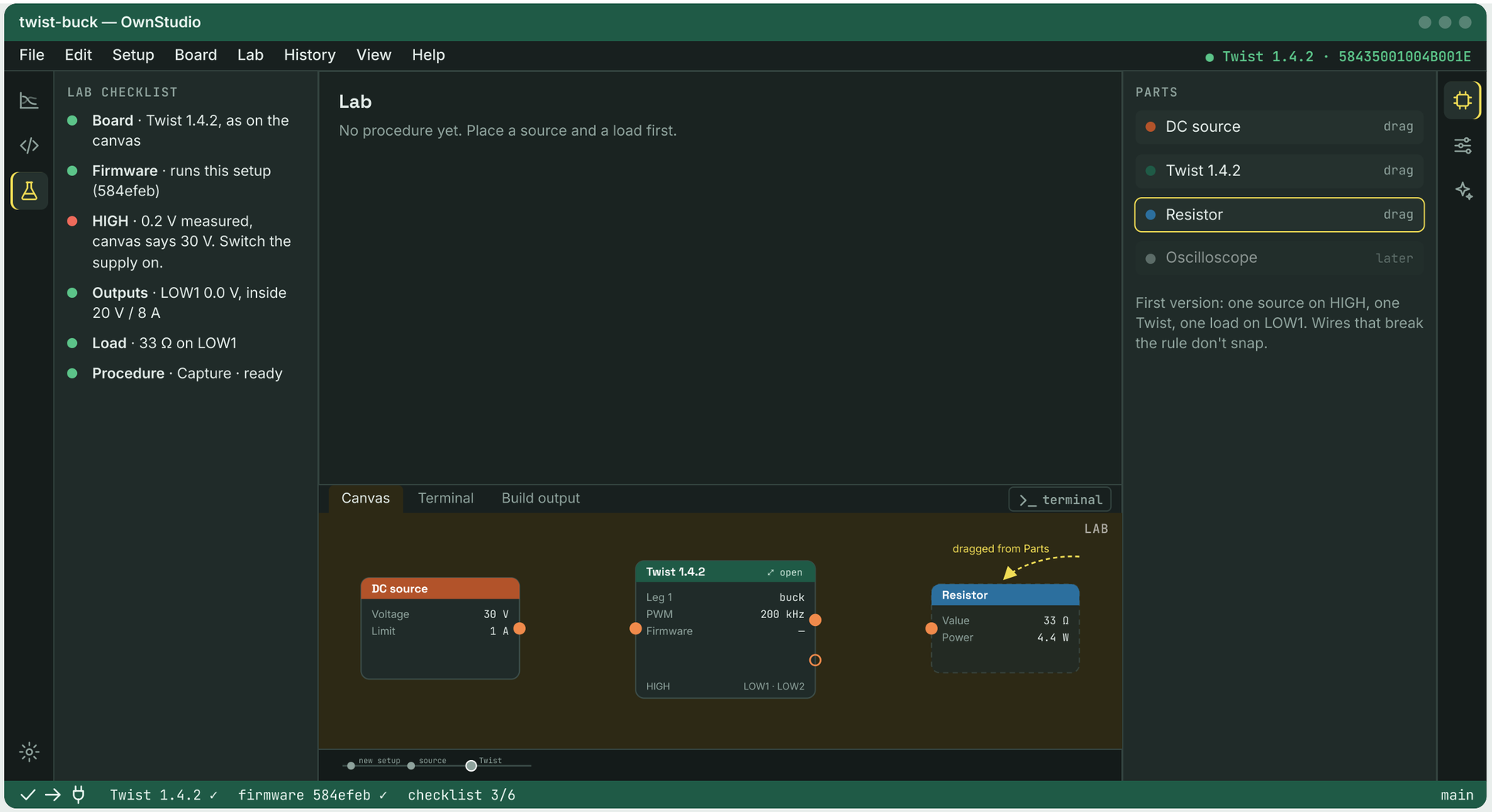
1

Start: choose a board

OBJECTIVE: BEFORE STEP 1

slice 1

- Shown when no setup is open. Replaces Home.
- The Twist on USB is detected by its serial number and highlighted, as Arduino IDE 2 does.
- Picking a board installs or updates its board definition (devicetree + board description) and creates a setup: a clone of Core with an ownstudio/ folder.
- Custom board: your overlay plus a board description.



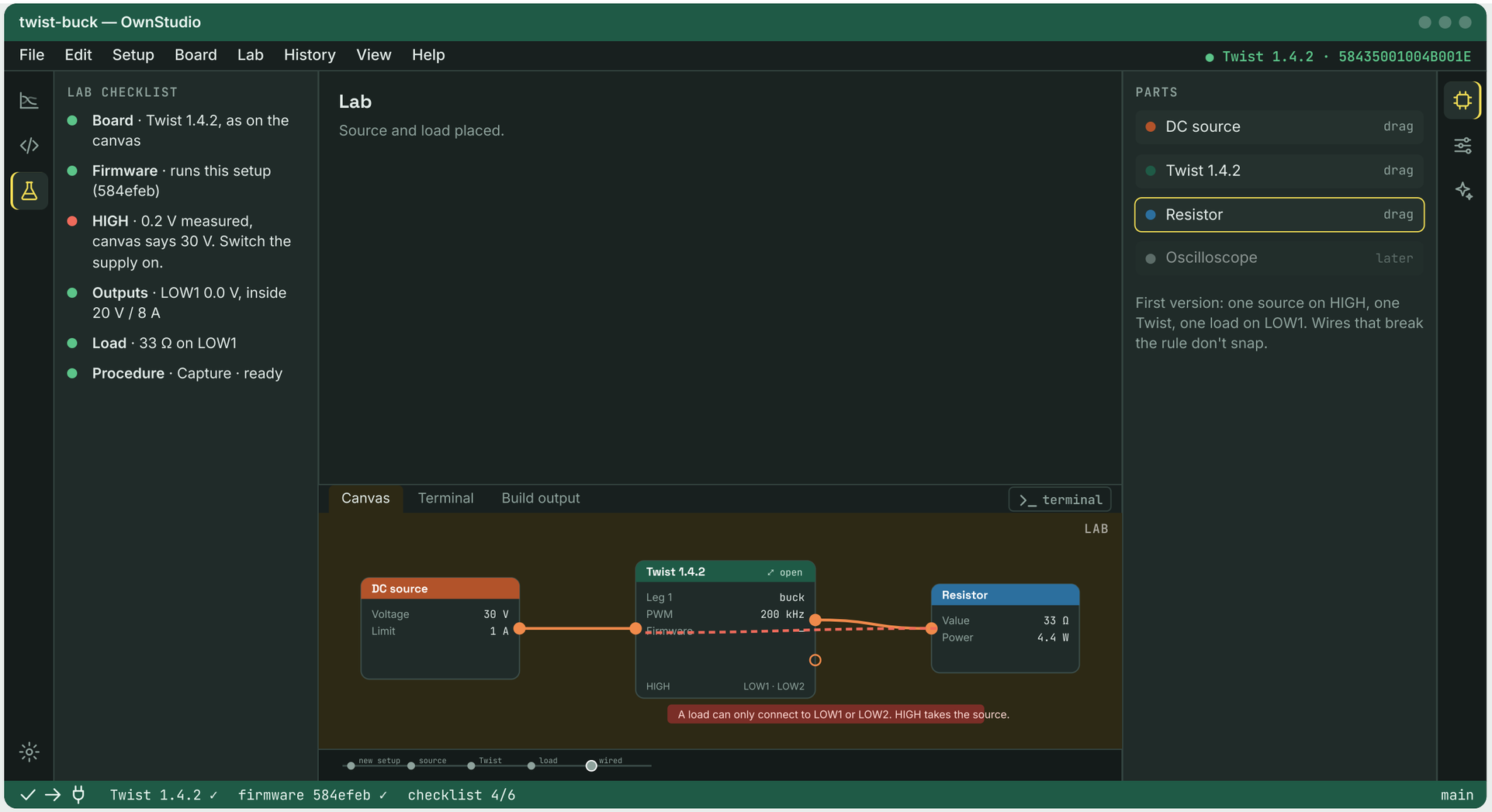
2

Drag the source, the Twist and the load

OBJECTIVE: STEP 1

`slice 1`

- The new setup opens with the Twist on the canvas.
- Parts on the right; the resistor is being dragged in.
- Values sit on the nodes. Ports are typed: power wires only.



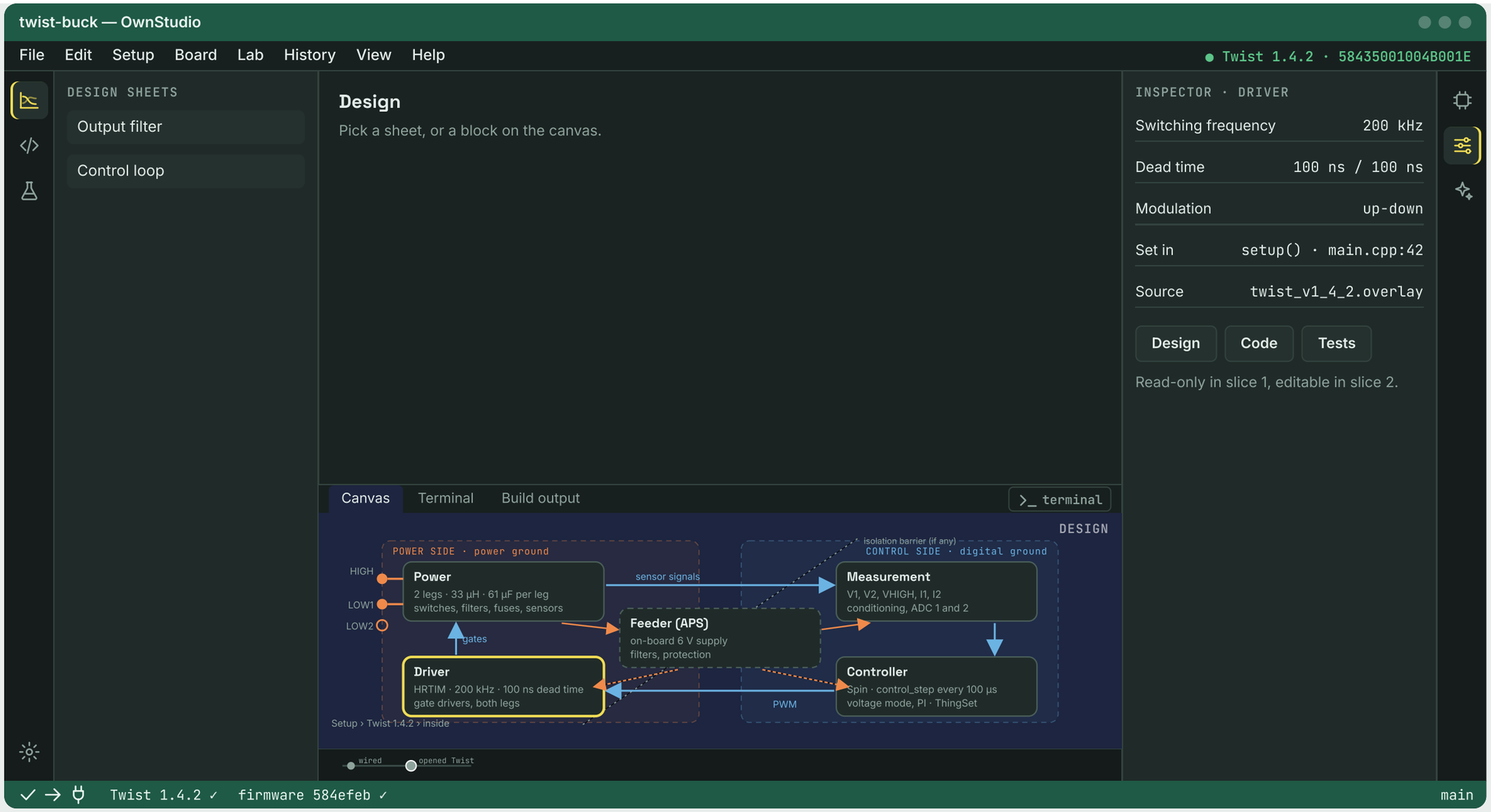
3

Wire them; a wrong wire does not snap

OBJECTIVE: STEP 1

`slice 1`

- Source → HIGH and LOW1 → load snap.
- A load dragged to HIGH is refused with one sentence.
- Each change becomes a dot on the timeline after 10 s without edits.



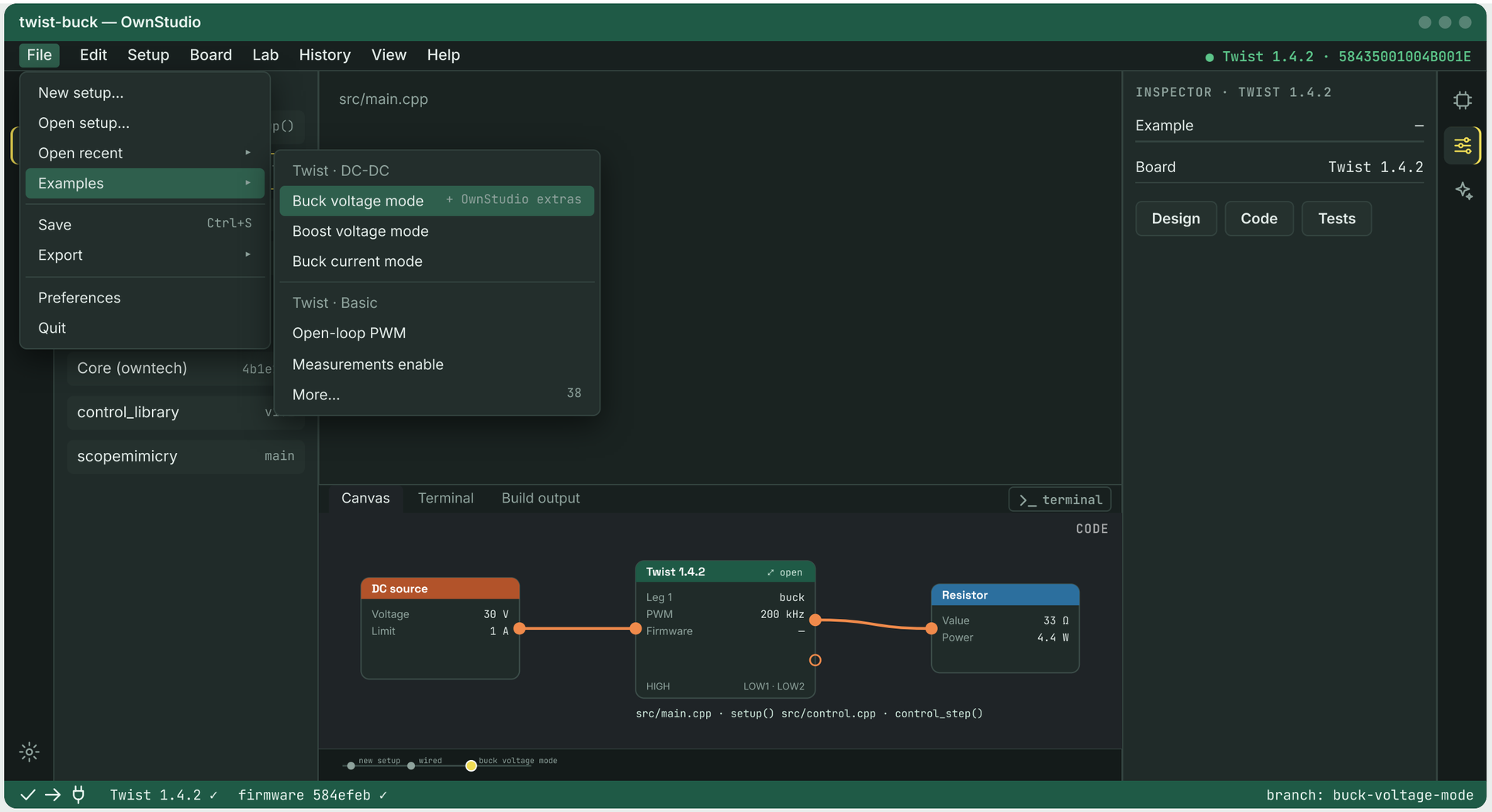
4

Open the Twist and look at its parts

OBJECTIVE: STEP 2

slice 1 slice 2

- Double-click the Twist: its five blocks on Luiz's reference layout: power side and control side, the feeder in the centre, signals looping clockwise, the isolation barrier between the grounds.
- Values come from the devicetree; ports HIGH, LOW1, LOW2 sit on the power block.
- Selecting a block fills the inspector: here the driver, 200 kHz and 100 ns from twist_v1_4_2.overlay.
- Slice 1 shows the settings; slice 2 makes them editable, written into setup() on save.
- Design, Code and Tests open what belongs to the block.



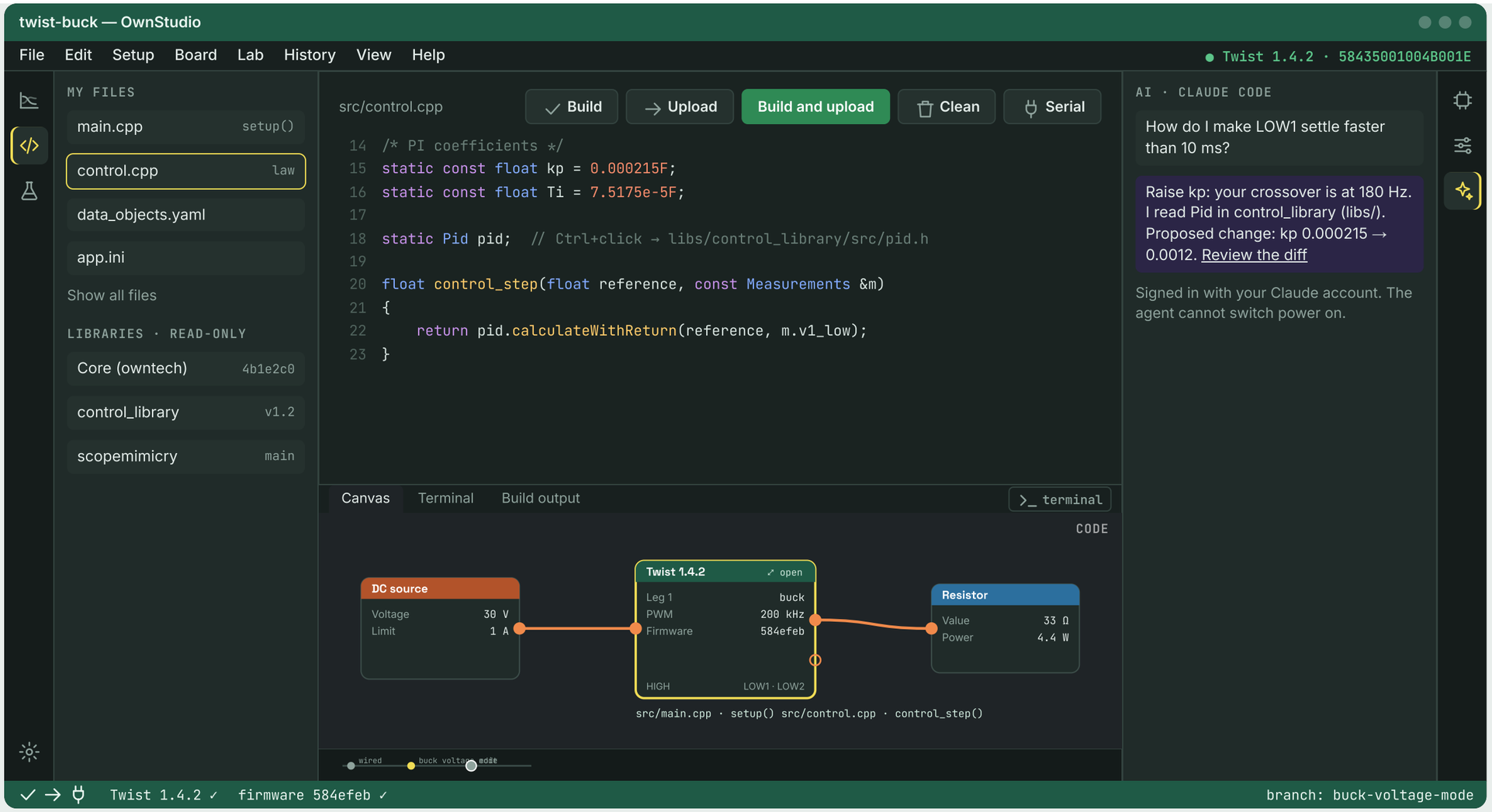
5

Load an example: it lands on its own branch

OBJECTIVE: STEP 3

slice 1

- File → Examples lists the Twist's examples from the OwnTech examples index.
- Loading copies the example into src/ on a new branch named after it, records its origin; nothing goes to an old/ folder.
- The timeline shows the branch point.



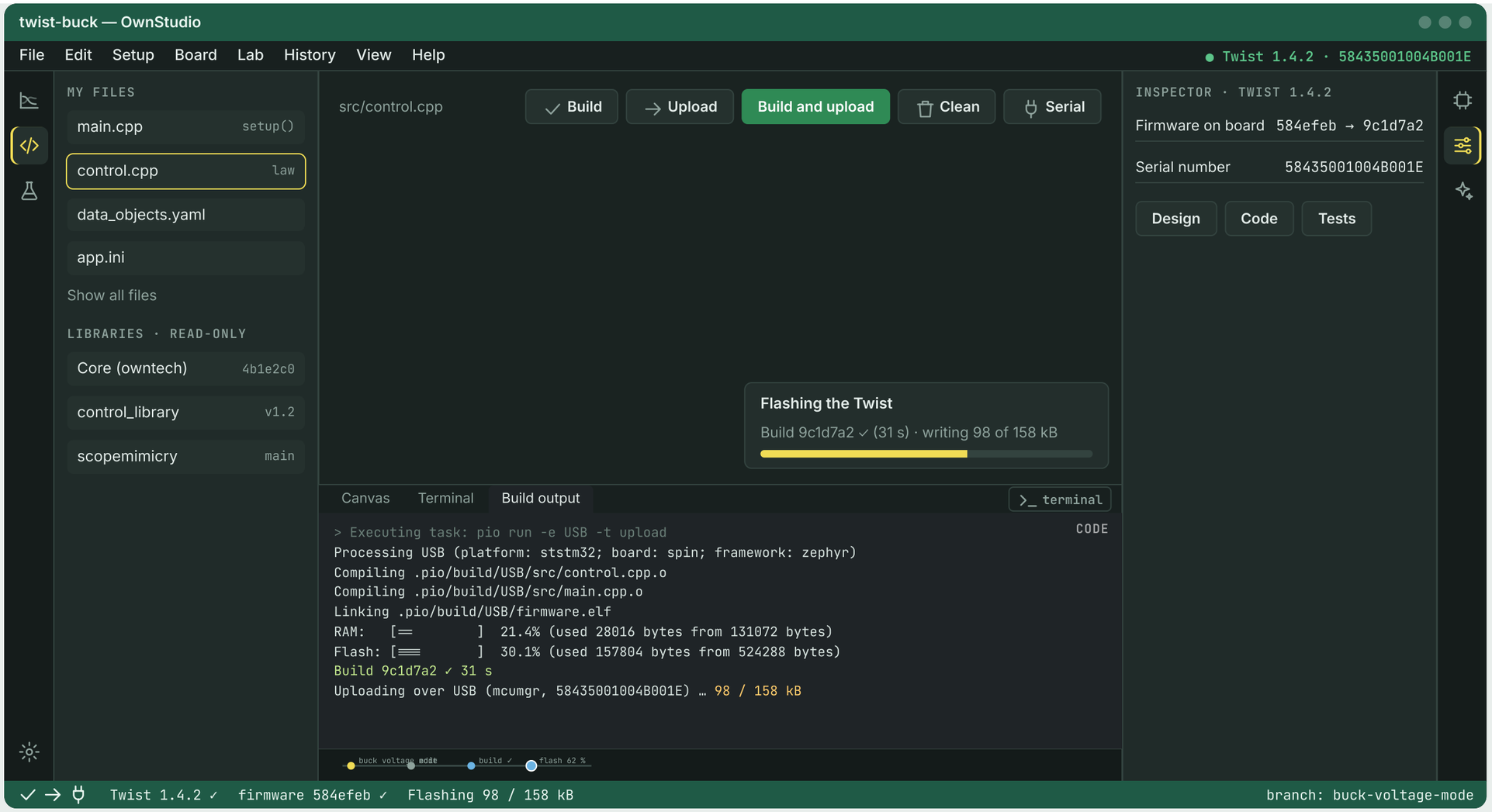
6

Code: the student's files, with real language support

OBJECTIVE: STEP 9 (CHANGE THE CODE)

slice 1

- A toolbar above the editor: Build, Upload, Build and upload, Clean, Serial monitor (as PlatformIO's); the same Build, Upload and Serial icons sit in the status bar in every workbench.
- My files first: main.cpp (its setup() is shared with the canvas) and control.cpp. Libraries below, read-only, so the user and the AI can read Core.
- Colouring, completion and Ctrl+click from clangd and basedpyright through the plugin system.
- The canvas, in its Code tint, says which file each block maps to.



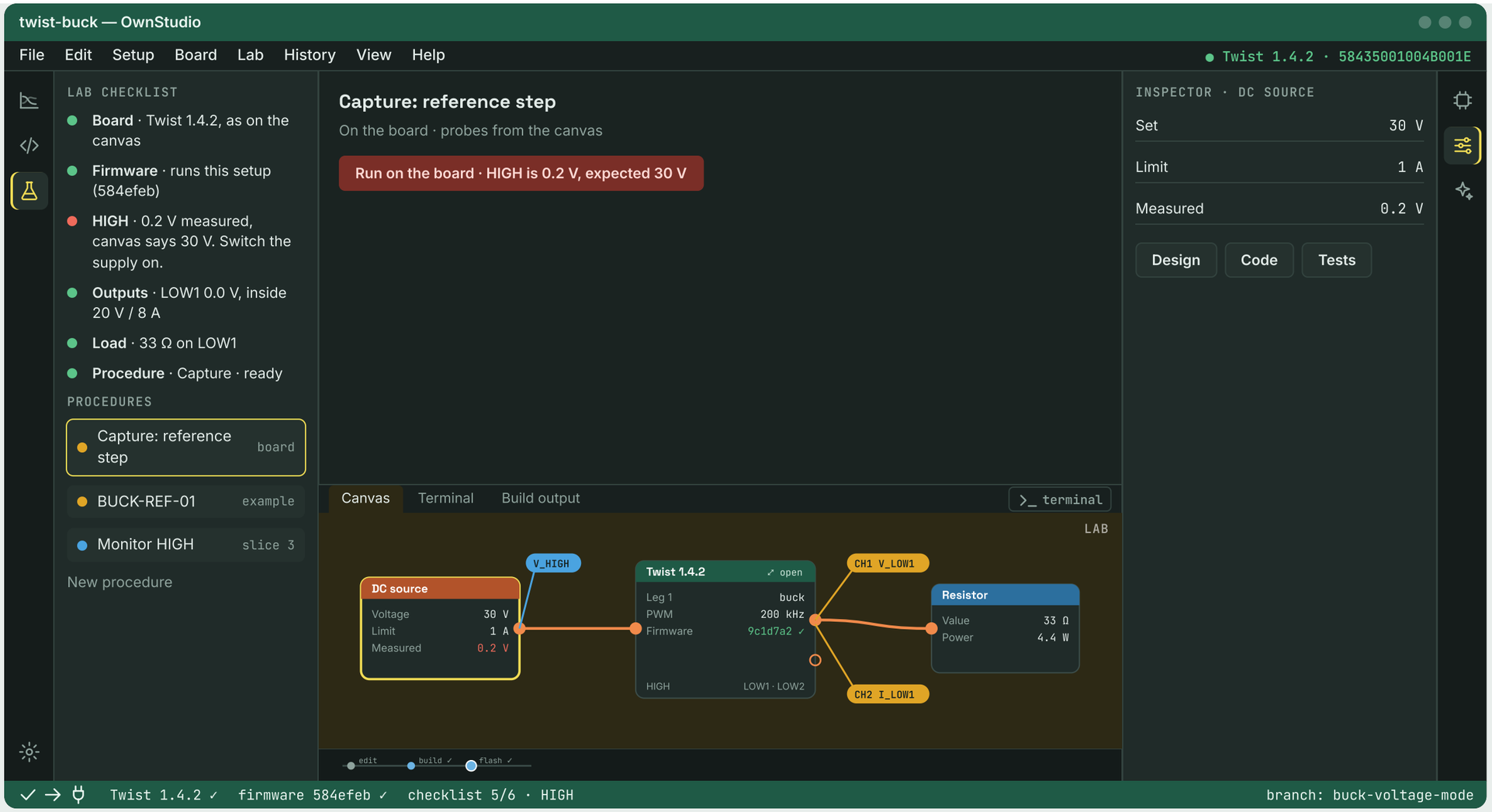
7

Build and flash, with real progress

OBJECTIVE: STEP 4

slice 1

- Build and upload from the Code toolbar, the status bar or the Board menu.
- The bottom area switches to Build output: PlatformIO's own log, as in VS Code, next to Canvas and Terminal.
- Compile steps, then bytes flashed out of the total: no back-and-forth bar.
- Both events become dots; the build identity ties every later record to this commit.



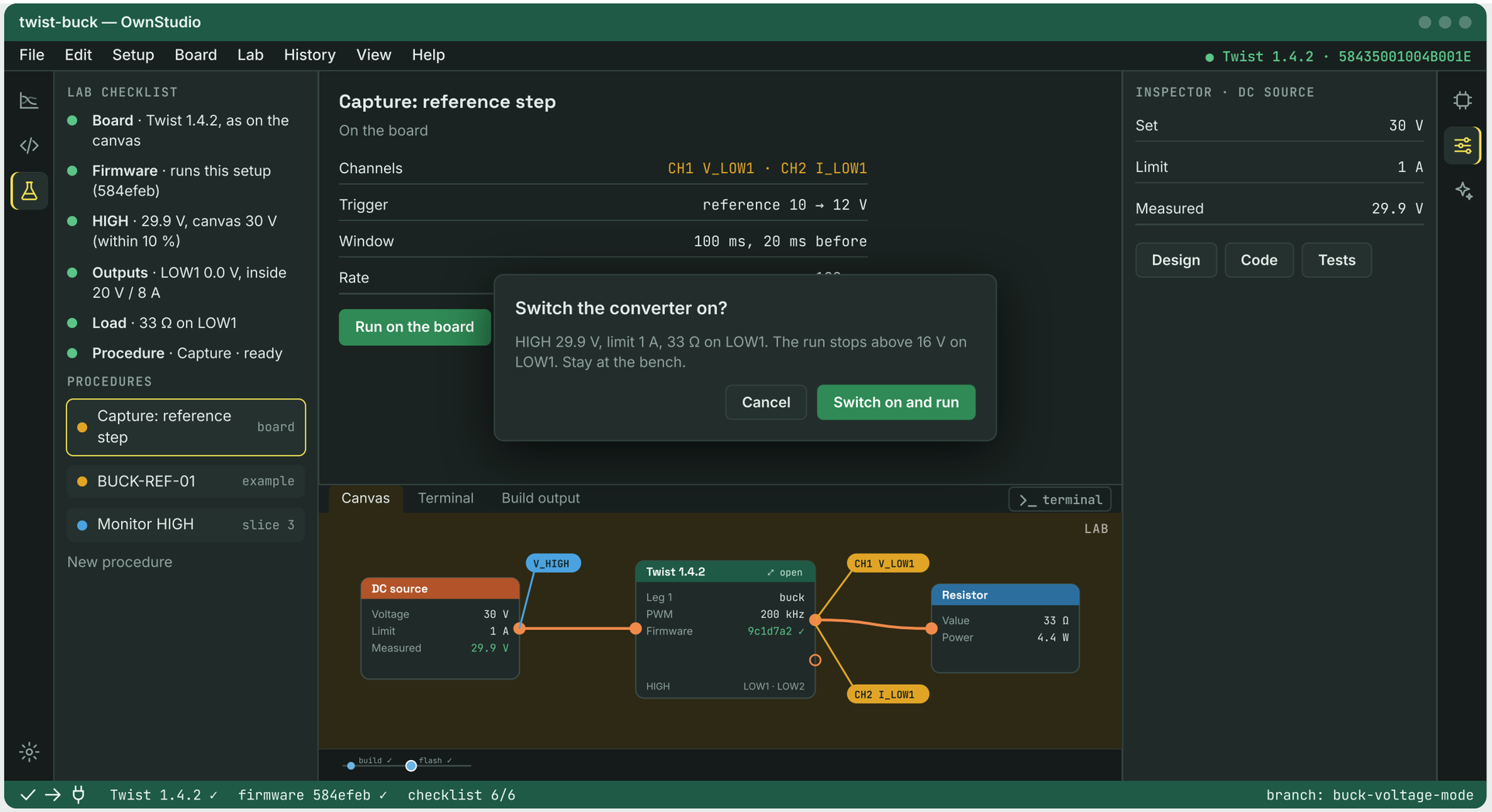
8

The Lab checks before Run turns green

OBJECTIVE: STEP 5

`slice 1`

- The checklist runs on its own and stays visible.
- Here HIGH reads 0.2 V while the canvas says 30 V: Run is red and says why.
- A bare Spin, the wrong firmware or limits crossed would turn it red the same way.



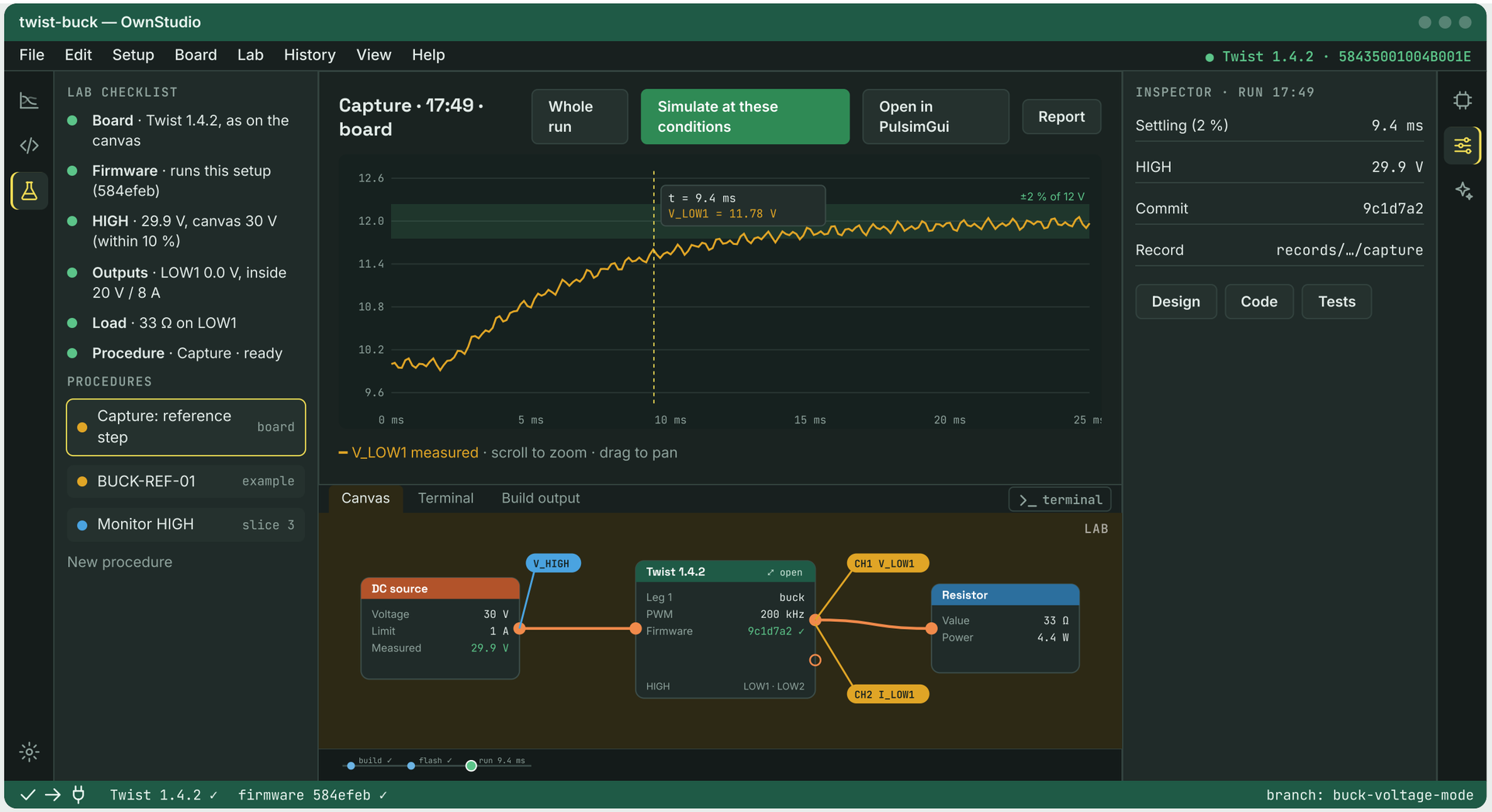
9

A simple procedure: switch on and capture

OBJECTIVE: STEP 5

slice 1 slice 3

- The board offers a default capture: probes from the canvas (amber = Capture, fast), a reference step, a duration.
- Supply on: every line is green, Run is green.
- Power always asks, and only OwnStudio's dialog issues the token the runner needs.
- Slice 3 replaces the form with Blockly test sequences.



10

The result: interactive, zoomed on the step

OBJECTIVE: STEP 5

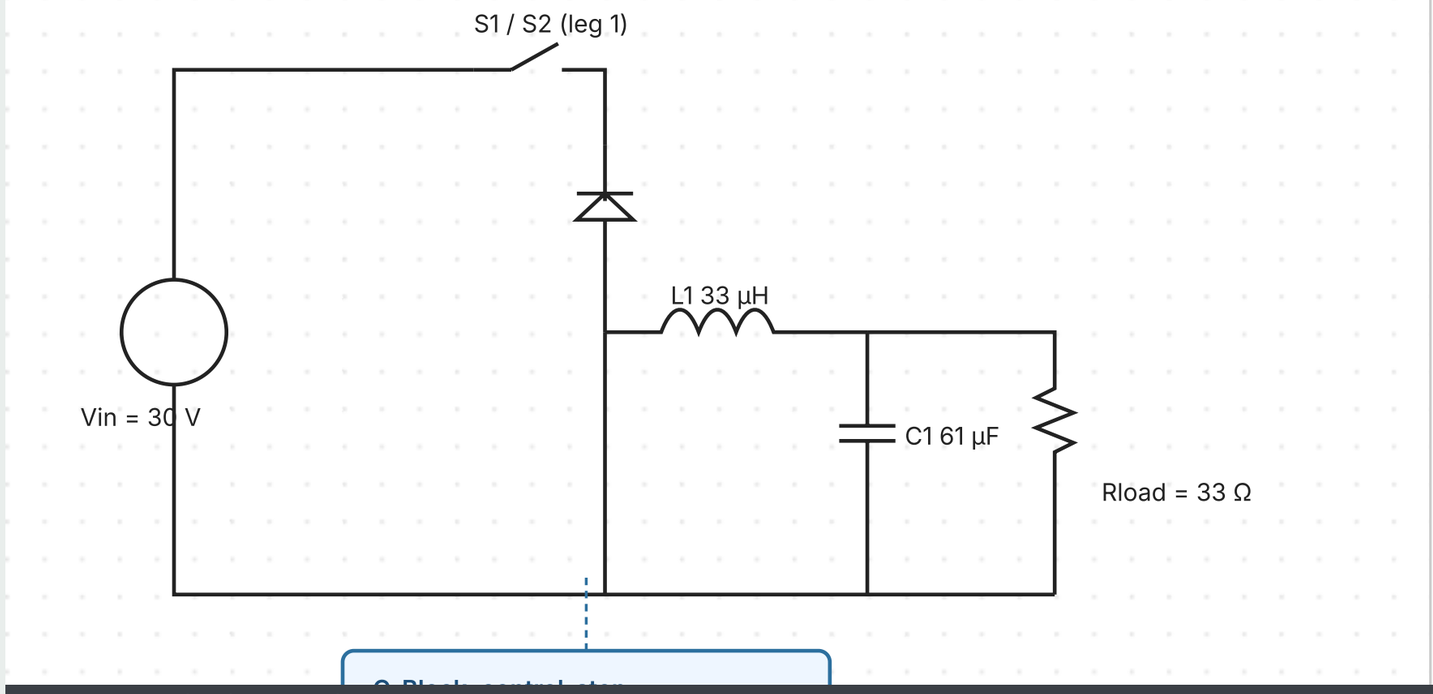
`slice 1`

- Opens on the event (the 10 → 12 V step), the whole run one click away.
- Zoom, pan, cursor readout, channels on and off.
- The run is a dot on the timeline; its light report goes to reports/.
- Next: Simulate, or Open in PulsimGui.

PulsimGui — twist_buck.pulsim (opened by OwnStudio)

File Edit View Simulation Help

Backend: OwnStudio runner

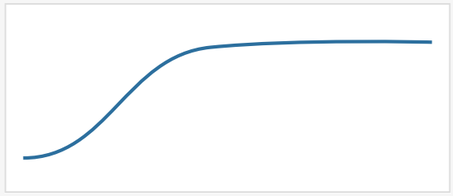


Properties · Rload

Resistance 33 Ω

Name "Rload" is synced with the canvas resistor.

Waveforms



Run uses the same runner as OwnStudio.

Saved · OwnStudio will sync Vin, Rload, L1, C1

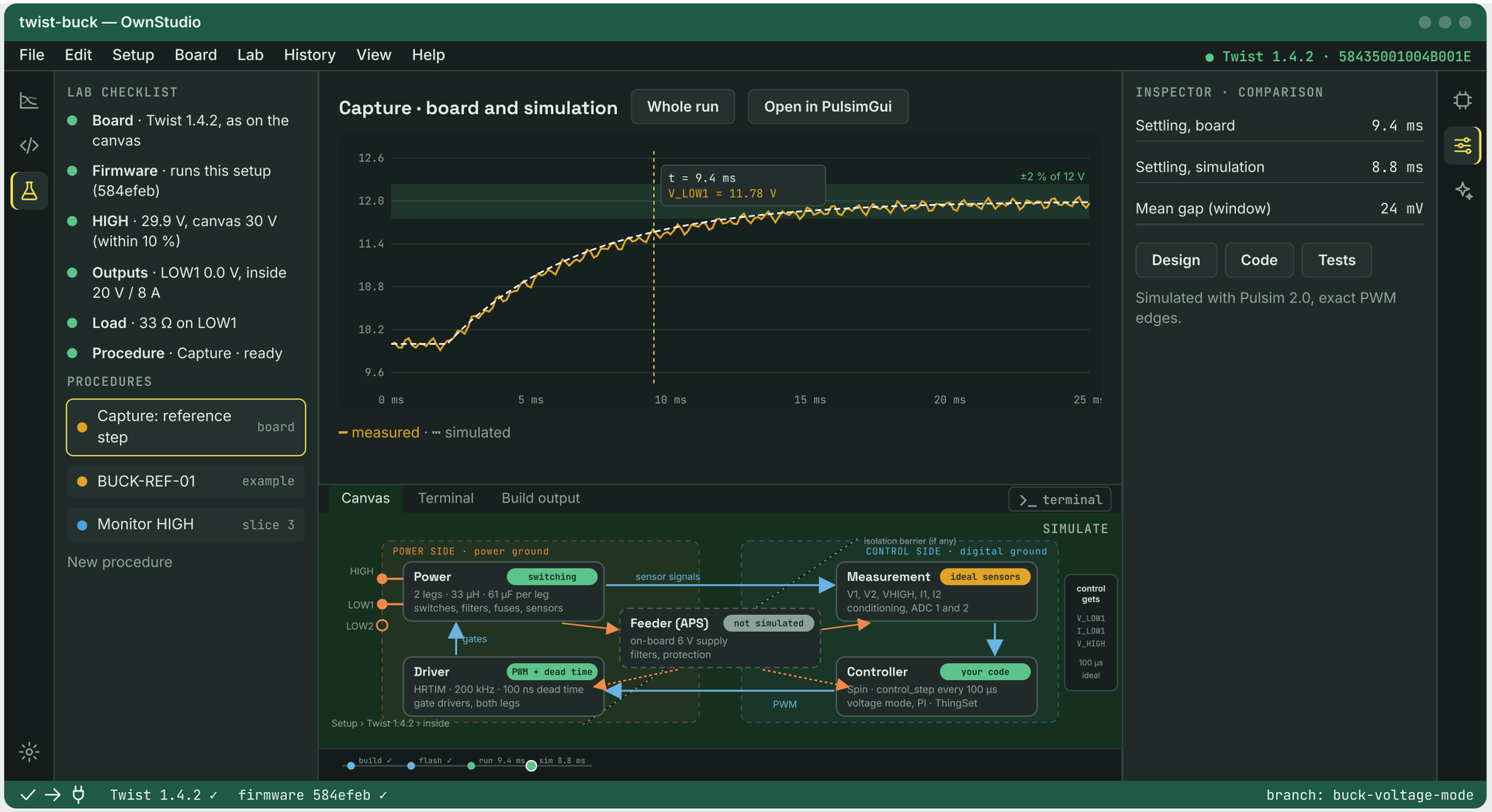
11

Open in PulsimGui: the simulator's circuit

OBJECTIVE: STEP 6

slice 1

- PulsimGui opens in its own window on the setup's .pulsim plant, with the canvas values already in it (30 V, 33 Ω) and the control code as a C-Block, as in Core2.
- Its Run button calls OwnStudio's runner: identical results.
- Save in PulsimGui writes the setup's file; OwnStudio syncs the named values back to the canvas.



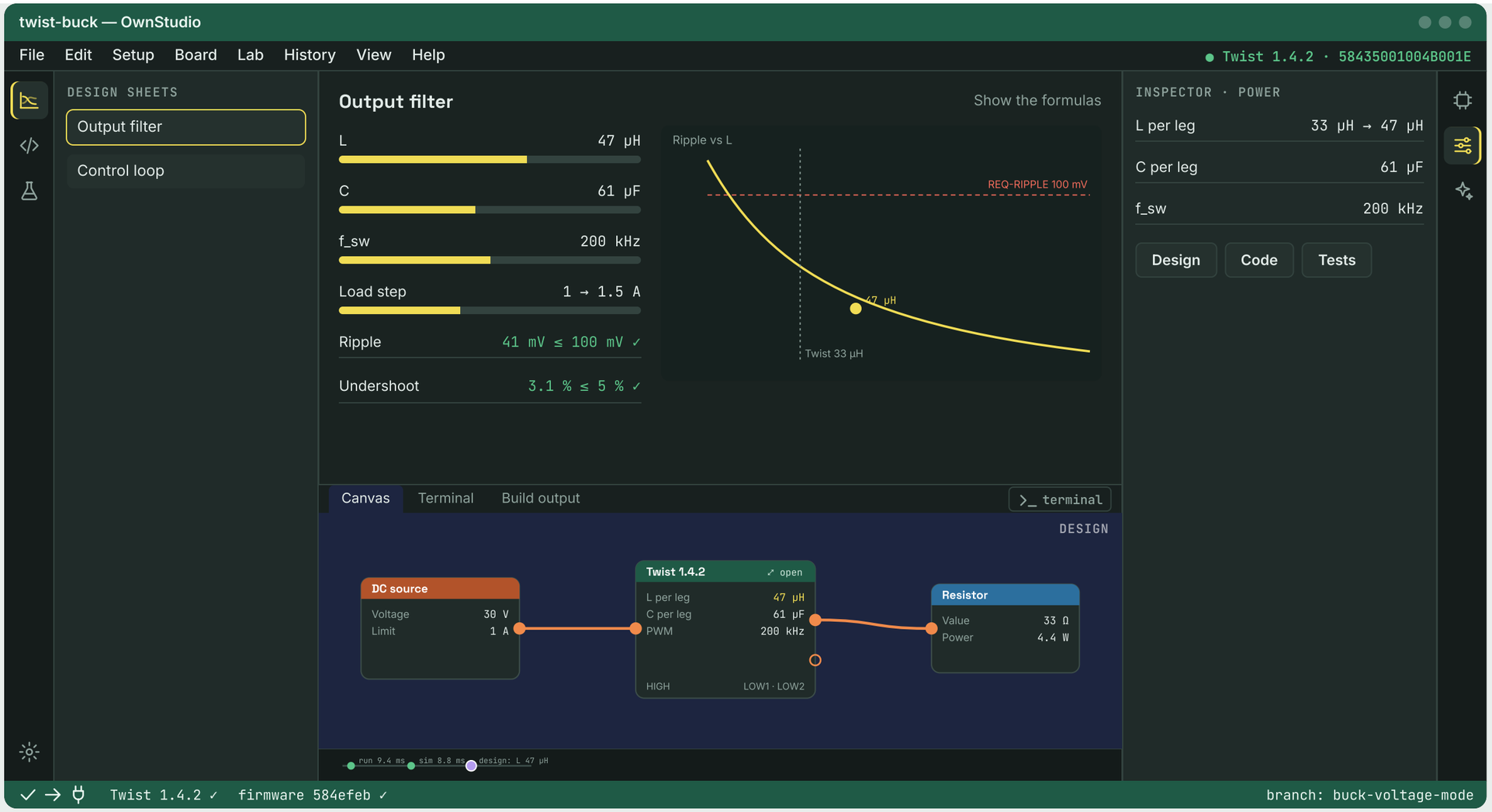
12

Simulate: the simulation over the measurement

OBJECTIVE: STEP 7

slice 1 slice 2

- "Simulate at these conditions" reruns the same procedure on the simulator with the canvas values (and the measured HIGH).
- White dashed: simulation; amber: measurement.
- The canvas in its Simulate tint shows fidelity chips (slice 2): what the simulation covers and what it leaves out.



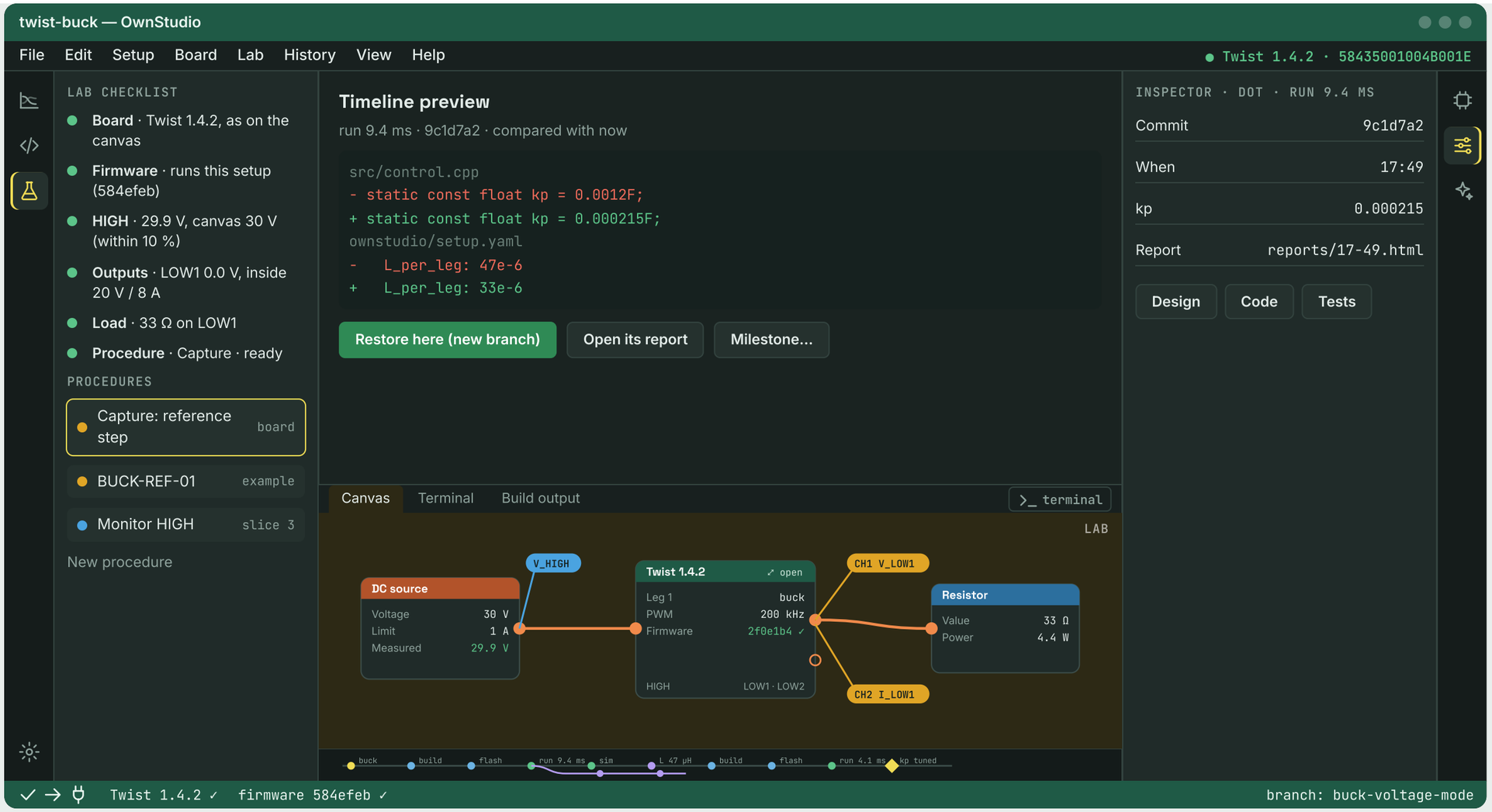
13

Design: change a parameter on a live sheet

OBJECTIVE: STEP 8

`slice 1`

- Sheets are forms, not code: sliders and fields, curves update live against the requirement lines; the Twist's own values are marked.
- Python still computes every number; "Show the formulas" opens the script.
- Changing L updates the canvas and the plant model; "Apply to the code" writes the gains.



14

Run again; go back in time; branch

OBJECTIVE: STEP 9

slice 1 slice 2

- Every run, build and change is a dot. A click previews that state; "Restore here" goes back without losing anything (it starts a branch).
- Branches are supported in slice 1; their lanes appear in slice 2 (purple, below).
- Records stay out of git; reports and the commit reproduce any dot.

Menu map — SPEC §5

File

- New setup...
- Open setup...
- Open recent ▸
- Examples ▸
- Save
- Save all
- Export ▸ report · wiring diagram
- Preferences
- Quit

Edit

- Undo
- Redo
- Cut
- Copy
- Paste
- Find
- Replace

Setup

- Add part ▸
- Libraries... slice 3
- Check setup
- Wiring diagram slice 2
- Simulation sheet slice 2
- Open in PulsimGui

Board

- Select board...
- Boards manager...
- Build
- Flash
- Build and flash
- Board info

Lab

- Run procedure
- Stop
- Run on the simulator
- New procedure
- AI takeover slice 4, when enabled
- Reports

History

- Milestone...
- Restore...
- New branch...
- Pull
- Push
- Remotes... slice 3
- Propose upstream... slice 3

View

- Design
- Code
- Lab
- Parts
- Inspector
- AI
- Canvas
- Timeline
- Terminal
- Zoom ▸

Help

- Tutorials
- Documentation
- Board datasheet
- Report a problem
- About OwnStudio

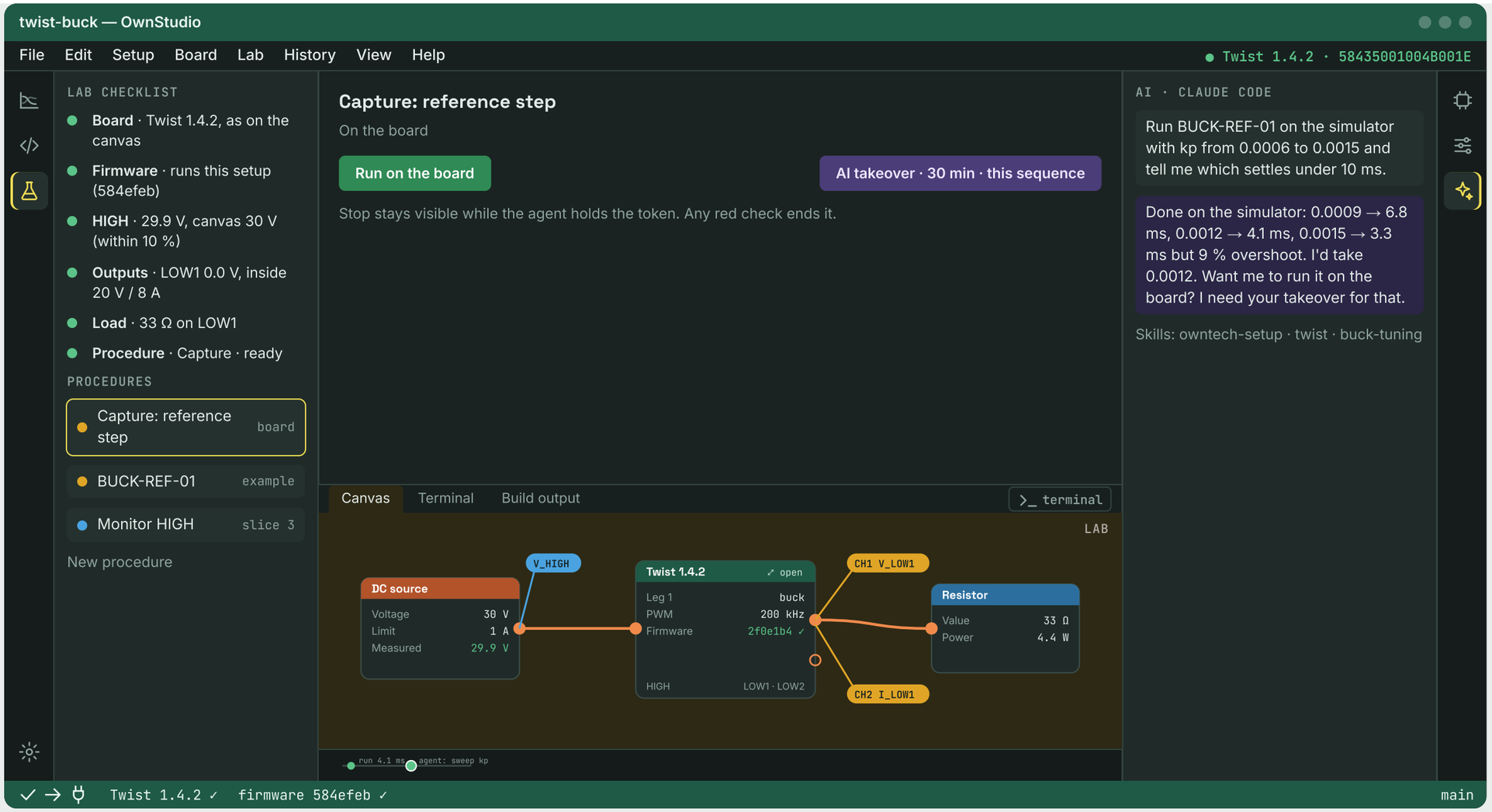
15

The eight menus

OBJECTIVE: EVERYWHERE

`slice 1`

- Theia's Selection, Go and Terminal menus are gone; the terminal is in View and has a button at the bottom.
- Every item has one owner module in the code.
- Items whose feature comes later say which slice brings it.



16

The AI: your own agent, and the takeover button

OBJECTIVE: AROUND EVERY STEP

slice 1 slice 4

- Claude Code or Codex, signed in with your own account; OwnTech skills come with the board, Core is its reference.
- The agent edits code and runs the simulator freely; it cannot switch power on.
- AI takeover (slice 4): enabled in the settings, apart from Run, in its own colour; a token for one sequence, 30 minutes, revoked on a red check, a change or Stop.